

ナップサック問題における総当たり法と動的計画法の 比較

氏名：玉城 俐空

2026 年 6 月 26 日

目次

1	内容	3
2	問題設定	3
3	総当たり法	3
4	動的計画法	4
5	総当たり法と動的計画法の比較	6
6	考察	7
7	まとめ	7

1 内容

本レポートでは、ナップサック問題を対象として、総当たり法と動的計画法（DP）による解法を実装し、それぞれの考え方と計算量について考察する。

ナップサック問題とは、複数の品物について、それぞれ重さと価値が与えられているとき、重さの上限を超えない範囲で価値の合計が最大となるような品物の組み合わせを求める問題である。

2 問題設定

本課題では、以下の重さと価値をもつ品物について、重さの合計が 45 以下となる範囲で価値の合計が最大となる組み合わせを求める。

Listing 1 問題で用いるデータ

```
1 Weight = [4,8,3,5,9,2,3,1,5,2,4,2,7,10,3,13,11,8,5,2,4,5,6]
2 money   = [6,12,4,3,7,1,3,2,7,3,4,2,10,13,5,16,14,9,1,4,6,2,4]
3
4 W = 45
```

ここで、Weight は各品物の重さ、money は各品物の価値、W はナップサックに入れられる重さの上限を表す。

3 総当たり法

総当たり法では、各品物について「選ぶ」または「選ばない」の 2 通りをすべて試す。品物の数を n とすると、全ての組み合わせの数は

$$2^n$$

である。

今回の問題では品物数が 23 個であるため、調べる組み合わせ数は

$$2^{23}$$

通りとなる。

Listing 2 総当たり法による実装

```
1 import time
2
3 start = time.perf_counter()
4
5 Weight = [4,8,3,5,9,2,3,1,5,2,4,2,7,10,3,13,11,8,5,2,4,5,6]
```

```

6 money = [6,12,4,3,7,1,3,2,7,3,4,2,10,13,5,16,14,9,1,4,6,2,4]
7
8 W = 45
9 n = len(Weight)
10
11 max_money = 0
12
13 for bit in range(2 ** n):
14     total_weight = 0
15     total_money = 0
16
17     for i in range(n):
18         if bit & (1 << i):
19             total_weight += Weight[i]
20             total_money += money[i]
21
22     if total_weight <= W:
23         if total_money > max_money:
24             max_money = total_money
25
26 end = time.perf_counter()
27
28 print("最大金額:", max_money)
29 print("実行時間:", end - start)

```

この方法では、すべての組み合わせを調べるため、必ず正しい答えを得ることができる。しかし、品物数が増えると組み合わせ数が指数関数的に増えるため、計算時間が大きくなるという欠点がある。

総当たり法の計算量は、

$$O(n2^n)$$

である。

4 動的計画法

動的計画法では、同じ計算を何度も行わないように、途中結果を配列に保存する。

今回の問題では、配列 `dp` を用いて、

$$dp[w] = \text{重さ } w \text{ 以下で達成できる最大価値}$$

と定義する。

例えば、 $dp[37] = 50$ であれば、「重さ 37 以内で達成できる最大価値が 50 である」という意味である。

各品物について、「その品物を入れない場合」と「その品物を入れる場合」を比較する。

$$dp[w] = \max(dp[w], dp[w - Weight[i]] + money[i])$$

ここで、 $dp[w]$ は品物を入れない場合、 $dp[w - Weight[i]] + money[i]$ は品物を入れる場合を表す。

Listing 3 動的計画法による実装

```
1 import time
2
3 start = time.perf_counter()
4
5 Weight = [4,8,3,5,9,2,3,1,5,2,4,2,7,10,3,13,11,8,5,2,4,5,6]
6 money = [6,12,4,3,7,1,3,2,7,3,4,2,10,13,5,16,14,9,1,4,6,2,4]
7
8 W = 45
9 dp = [0] * (W + 1)
10 n = len(Weight)
11
12 for i in range(n):
13
14     for w in range(W, Weight[i] - 1, -1):
15
16         # 品物を入れない
17         no_item = dp[w]
18
19         # 品物を入れる
20         yes_item = dp[w - Weight[i]] + money[i]
21
22         # 良い方を採用
23         if yes_item > no_item:
24             dp[w] = yes_item
25
26 end = time.perf_counter()
27
28 print("最大金額:", dp[W])
29 print("実行時間:", end - start)
```

この方法では、重さごとの最大価値を保存しながら計算するため、同じ部分問題を何度も解く必要がない。

動的計画法の計算量は、

$$O(nW)$$

である。

今回の場合、品物数は 23 個、重さの上限は 45 であるため、処理回数はおよそ

$$23 \times 45$$

となる。

5 総当たり法と動的計画法の比較

総当たり法では、すべての品物の組み合わせを調べるため、計算量は $O(2^n)$ となる。一方、動的計画法では、重さごとの最大価値を保存しながら計算するため、計算量は $O(nW)$ となる。

表 1 総当たり法と動的計画法の比較

手法	考え方	計算量
総当たり法	すべての組み合わせを調べる	$O(2^n)$
動的計画法	途中結果を保存して再利用する	$O(nW)$

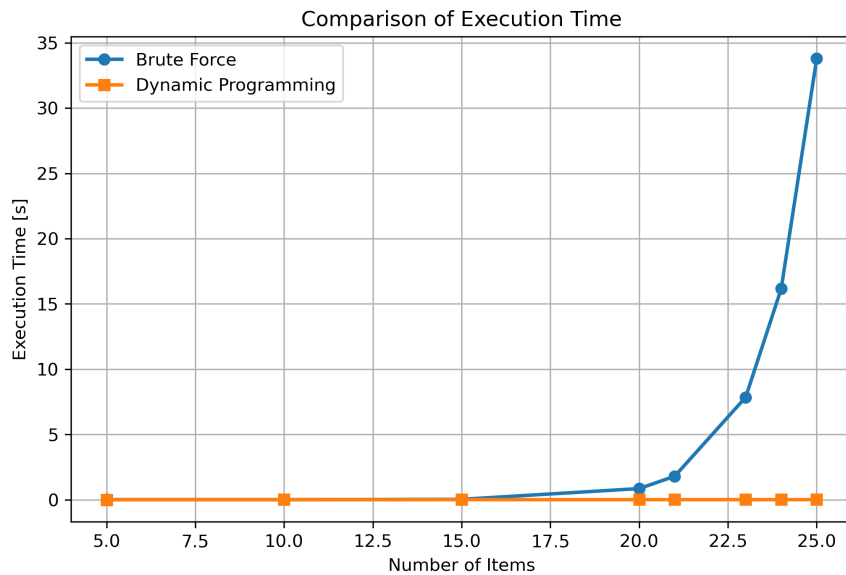


図 1 総当たり法と動的計画法の実行時間比較

図 1 より、品物数が増えるほど総当たり法は急激に計算量が増えることが分かる。これに対して動的計画法は、重さの上限が大きすぎなければ効率よく解を求めることができる。

6 考察

総当たり法は、すべての組み合わせを調べるため、アルゴリズムとしては単純で理解しやすく、必ず最適解を得ることができる。しかし、品物数が増えると組み合わせ数が 2^n 通りに増加するため、実行時間が長くなる。

一方、動的計画法では、 $dp[w]$ を「重さ w 以内で達成できる最大価値」として保存することで、同じ計算を繰り返さずに済む。そのため、総当たり法よりも効率的に解を求めることができる。

特に、 $dp[w - \text{Weight}[i]] + \text{money}[i]$ という式は、「現在の品物を入れた場合、残りの重さで達成できる最大価値に、現在の品物の価値を加える」という意味である。このように、動的計画法では小さな問題の答えを利用して、大きな問題の答えを求めている。

7 まとめ

本レポートでは、ナップサック問題に対して総当たり法と動的計画法を用いて解を求めた。総当たり法はすべての組み合わせを調べるため分かりやすいが、計算量が大きい。一方、動的計画法は途中結果を保存して再利用することで、より効率的に最大価値を求めることができる。